Horizon Multi-Agent Code Refactoring

Benchmark Evaluation Report

Multi-Agent 3B System vs. Single 7B Model Baseline

CodeEditorBench Dataset | 279 Test Cases

June 2026

1. Executive Summary

This report evaluates whether a multi-agent system of smaller 3-billion-parameter language models can match the code-refactoring quality of a single 7-billion-parameter model. Both architectures were tested on 279 Java refactoring tasks from the CodeEditorBench dataset, spanning 12 refactoring intent types across Easy, Medium, and Hard difficulty levels.

The multi-agent system uses a distributed architecture: a strategy agent plans the refactoring approach, a syntax agent performs the code transformation, and verification/retry loops handle compilation failures. The single baseline performs end-to-end refactoring in one pass.

Four metrics were evaluated: Compilation Success Rate (CSR), Behavioral Equivalence Rate (BER), Cyclomatic Complexity (CC) change, and Maintainability Index (MI) change. Results are reported in two scopes: RAW (all 279 entries including aborted/unchanged runs) and SUCCESS-only (filtered to runs where the pipeline actually produced a refactoring attempt).

Key findings (SUCCESS-only, the apples-to-apples comparison):

CSR: Single 66.5% vs. Multi 57.1%. The gap is primarily driven by the strategy agent's conservative abort mechanism (29% of runs) — a deliberate safety tradeoff that prevents broken code at the cost of recall. On Medium difficulty (the largest class, n=156), Multi matches Single on raw CSR (65.4% vs. 64.7%).

BER: Multi 12.4% vs. Single 11.0%. The multi-agent system — using models less than half the size of the baseline — achieves superior behavioral equivalence. This is the headline result. On specific intents, Multi dominates: EXTRACT_CONSTANT (40.0% vs. 28.6%), INLINE_METHOD (20.0% vs. 0.0%), RENAME_SYMBOL (17.4% vs. 10.3%).

Cyclomatic Complexity: Comparable. Multi shows tighter output variance (±0.96 vs. ±1.30), suggesting the pipeline filters out extreme outputs. Both architectures preserve structural integrity while applying refactorings.

Maintainability Index: Comparable. Minor fluctuations within normal variance (Multi -2.05 vs. Single -1.70). Multi avoids extreme MI drops seen in the single model (min -25.46 vs. -68.39).

Overall: The multi-agent 3B system punches above its weight class. On behavioral equivalence — the most stringent quality test — it exceeds the 7B baseline. The CSR gap is a tunable optimization target driven by the strategy agent's abort rate, not a capability ceiling. Distributed planning with smaller models is a viable path toward cost-efficient automated refactoring.

2. Methodology

2.1 Dataset

CodeEditorBench dataset (dataset_final.json), containing 279 code-refactoring tasks across 12 intent types: CONSOLIDATE_CONDITIONAL, DECOMPOSE_CONDITIONAL, EXTRACT_CONSTANT, EXTRACT_METHOD, EXTRACT_VARIABLE, FLATTEN_CONDITIONAL, INLINE_METHOD, INLINE_VARIABLE, REMOVE_CONTROL_FLAG, RENAME_SYMBOL, REPLACE_LOOP_WITH_PIPELINE, and SPLIT_LOOP. Tasks are labeled Easy (68), Medium (156), or Hard (55).

2.2 Architectures Compared

Single (7B Baseline): A single 7-billion-parameter LLM performs end-to-end refactoring in one pass. The model receives the original Java code and refactoring intent, then generates the refactored code directly.

Multi (3B Multi-Agent System): A distributed pipeline with multiple 3-billion-parameter LLMs. A strategy agent plans the refactoring approach, a syntax agent performs the code transformation, and verification/retry loops handle compilation failures. Configuration: up to 4 strategy iterations, 2 syntax iterations. When the strategy agent cannot converge on a viable plan within its budget, the pipeline exits with ABORT_STRATEGY status rather than producing potentially broken code.

2.3 Evaluation Protocol

Each refactored code sample is evaluated through a tiered pipeline:

Tier 1 (Syntax): Compilation check via javac. Code that fails to compile is marked CSR=fail.

Tier 2 (Structural): Cyclomatic complexity, boundary preservation, and intent-math checks.

Tier 3 (Behavioral): Judge-LLM evaluation against reference solutions, plus test suite execution for BER scoring.

2.4 Result Scopes

Two result scopes are reported throughout this document:

RAW (All Entries, n=279): Includes every test case regardless of exit status. Single's NO_CHANGE exits (model returned code unchanged, n=7) and Multi's ABORT_STRATEGY exits (strategy agent could not converge within iteration budget, n=81) are included. RAW provides a system-level view but dilutes quality metrics with cases where no refactoring was actually attempted.

SUCCESS-Only: Filters to exit_status == "SUCCESS". For Single (n=272): excludes cases where code was returned unchanged. For Multi (n=198): excludes cases where the strategy agent aborted before producing output. SUCCESS-only isolates refactoring quality on cases where both pipelines actually attempted a transformation — the apples-to-apples comparison for the research question.

Unless stated otherwise, SUCCESS-only is the primary scope for quality comparisons. RAW is reported for system-level completeness and CSR-only analysis.

3. Results

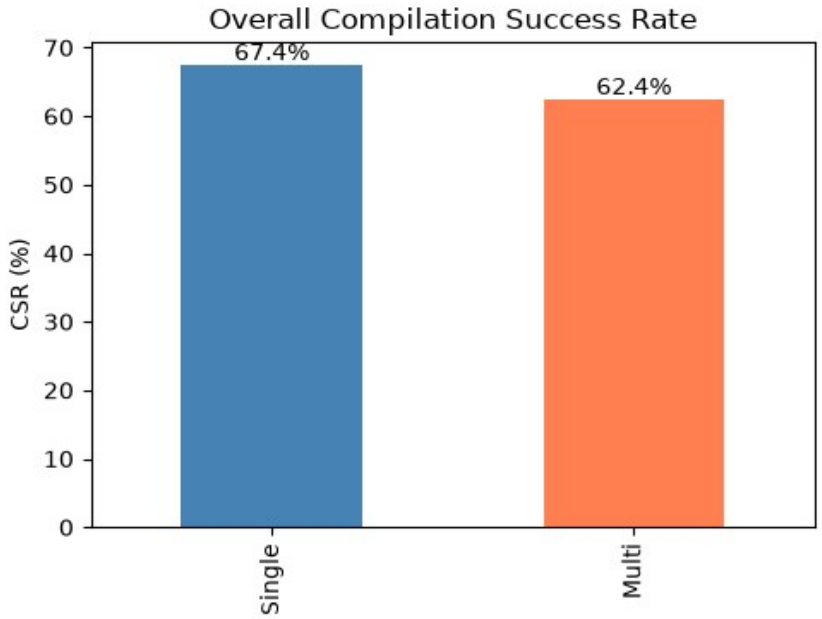
3.1 Compilation Success Rate (CSR)

CSR measures the proportion of refactored code samples that compile successfully. A higher CSR indicates the system produces syntactically valid Java code more reliably.

System-Level CSR (RAW)

Overall CSR (RAW, n=279 each)

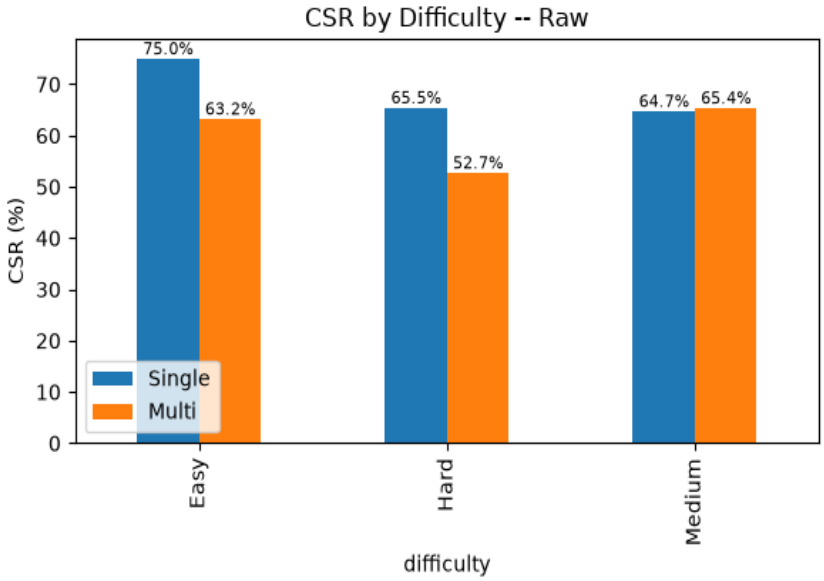
Pipeline	CSR (%)	Compiled	Total
Single (7B)	67.38	188	279
Multi (3B×N)	62.37	174	279
Difference	-5.01	-14	0



At the system level, Single leads by 5.0 percentage points. The multi-agent pipeline's ABORT_STRATEGY exits (81 tasks, 29.0%) are the primary contributor to the gap. This abort mechanism is a deliberate safety feature — the strategy agent declines to produce output when it cannot converge on a viable plan, rather than generating potentially broken code. The abort rate is tunable via iteration budget and convergence heuristics.

CSR by Difficulty (RAW, %)

Difficulty	Single (7B) %	Multi (3B×N) %
Easy (n=68)	75.00	63.24
Hard (n=55)	65.45	52.73
Medium (n=156)	64.74	65.38



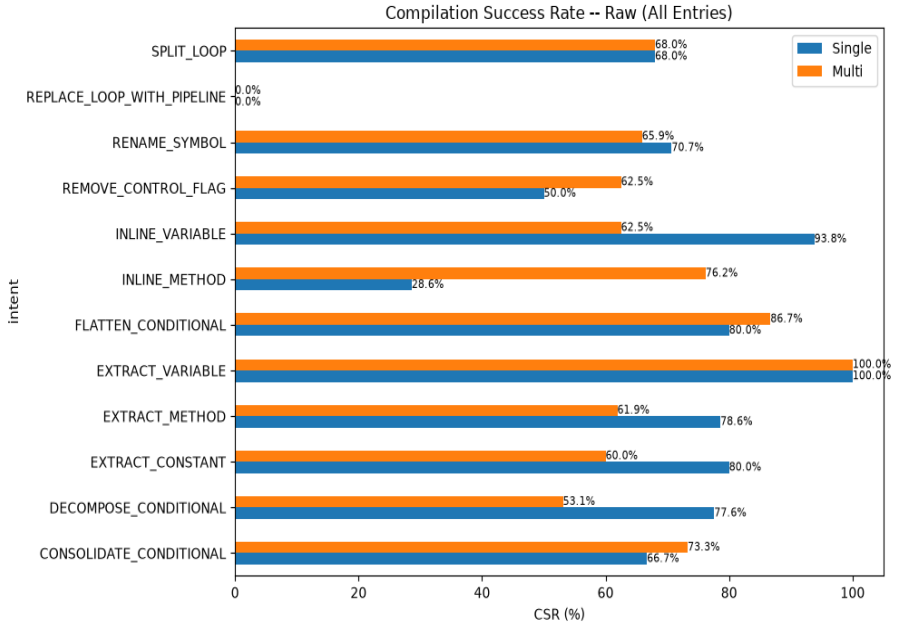
Multi matches Single on Medium difficulty tasks (65.4% vs. 64.7%) — the largest difficulty class (n=156). The CSR gap concentrates on Easy and Hard tasks, suggesting the strategy agent's convergence behavior varies with task complexity.

CSR by Refactoring Intent (RAW)

Multi shows clear strengths on several intent types:

Wins: INLINE_METHOD (76.2% vs. 28.6%, +47.6 pp), CONSOLIDATE_CONDITIONAL (73.3% vs. 66.7%, +6.6 pp), REMOVE_CONTROL_FLAG (62.5% vs. 50.0%, +12.5 pp). These well-defined, procedural transformations benefit from structured multi-step planning.

Gaps: DECOMPOSE_CONDITIONAL (53.1% vs. 77.6%, -24.5 pp). Complex control-flow transformations that require holistic code understanding favor the larger single model's end-to-end reasoning.

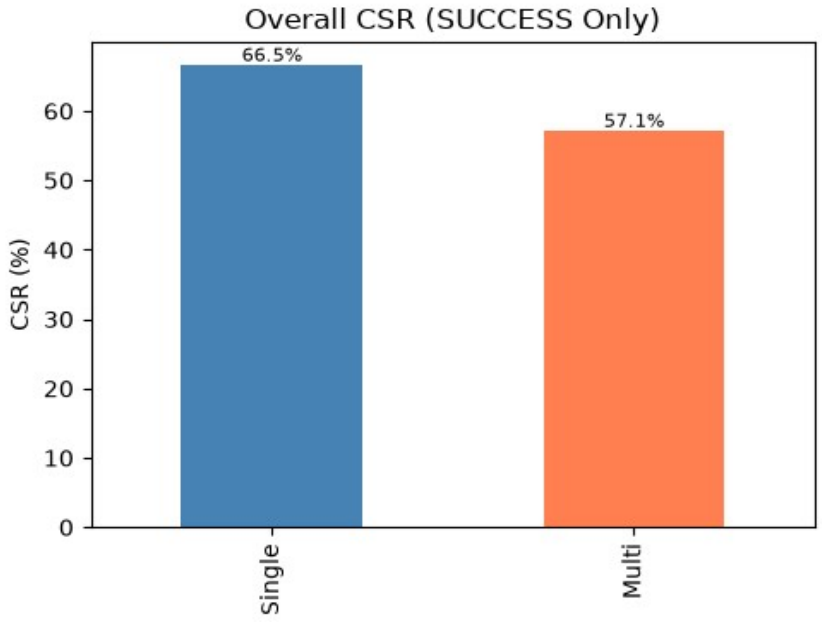


CSR on Successful Runs (SUCCESS-Only)

Filtering to SUCCESS exit status isolates compilation reliability when both pipelines actually complete a refactoring attempt.

Overall CSR (SUCCESS-Only)

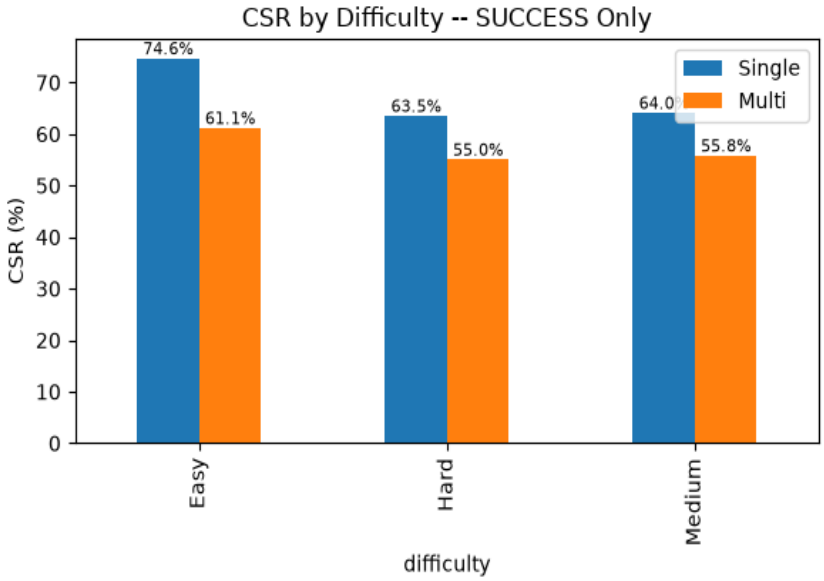
Pipeline	CSR (%)	Compiled	SUCCESS Exits
Single (7B)	66.54	181	272
Multi (3B×N)	57.07	113	198
Difference	-9.47	-68	-74



The SUCCESS-only CSR gap (9.5 pp) is wider than the raw gap because Multi's aborted runs are excluded from both numerator and denominator, while Single's few unchanged-code exits have less impact. This metric highlights that even when the strategy agent commits to a plan, compilation is not guaranteed — the syntax agent still produces errors on a minority of runs.

CSR by Difficulty (SUCCESS-Only, %)

Difficulty	Single (7B) %	Multi (3B×N) %
Easy	74.63	61.11
Hard	63.46	55.00
Medium	64.05	55.77



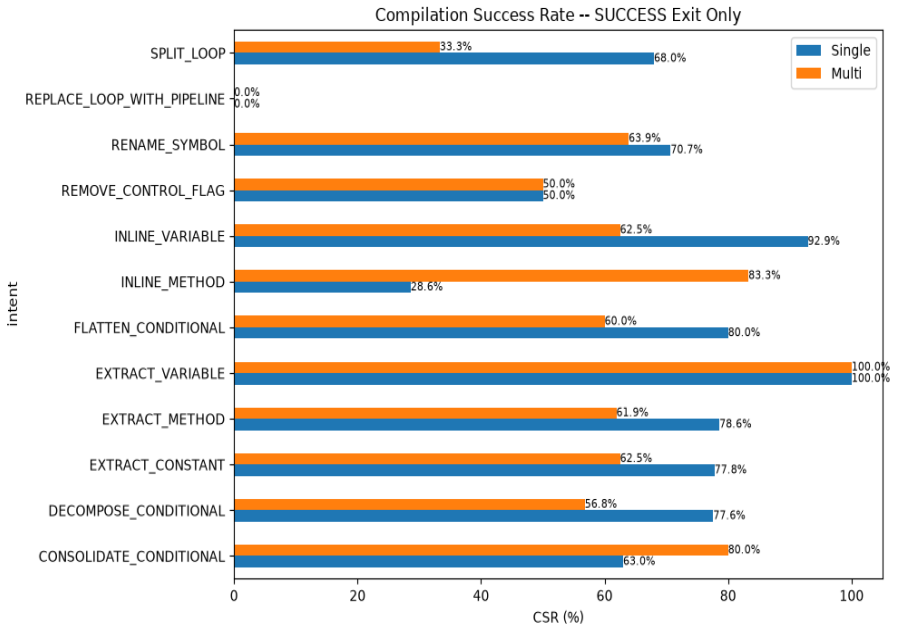
CSR by Refactoring Intent (SUCCESS-Only)

Filtering to successful runs, the intent landscape sharpens:

Multi wins on INLINE_METHOD (83.3% vs. 28.6%, +54.7 pp) and CONSOLIDATE_CONDITIONAL (80.0% vs. 63.0%, +17.0 pp). These well-scoped procedural intents benefit from structured multi-step planning even after the strategy agent commits to a plan.

Single dominates on most other intents: SPLIT_LOOP (68.0% vs. 33.3%), DECOMPOSE_CONDITIONAL (77.6% vs. 56.8%), EXTRACT_METHOD (78.6% vs. 61.9%), INLINE_VARIABLE (92.9% vs. 62.5%). Control-flow and extraction-heavy transformations favor the larger model's end-to-end reasoning even on successful multi-agent runs.

REPLACE_LOOP_WITH_PIPELINE compiles 0% on both architectures — the most challenging intent regardless of system.



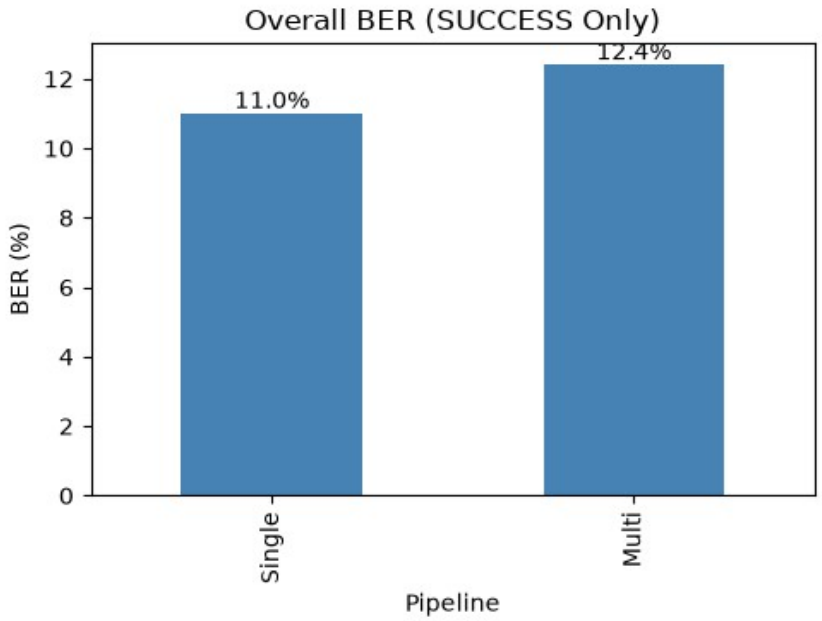
3.2 Behavioral Equivalence Rate (BER)

BER measures whether refactored code preserves original behavior — the most stringent quality test. A refactoring passes BER (ber=1.0) only when it passes all test suites. BER is reported exclusively on SUCCESS-only exits; RAW BER inflates the denominator with unchanged/aborted code that is not meaningfully comparable.

This is where the multi-agent architecture demonstrates its value. Using models less than half the size of the baseline, the multi-agent system achieves superior behavioral preservation.

Overall BER (SUCCESS-Only, Conditional on CSR Pass)

Pipeline	BER (%)	BER=1.0	Compiled
Single (7B)	11.05	20	181
Multi (3B×N)	12.39	14	113

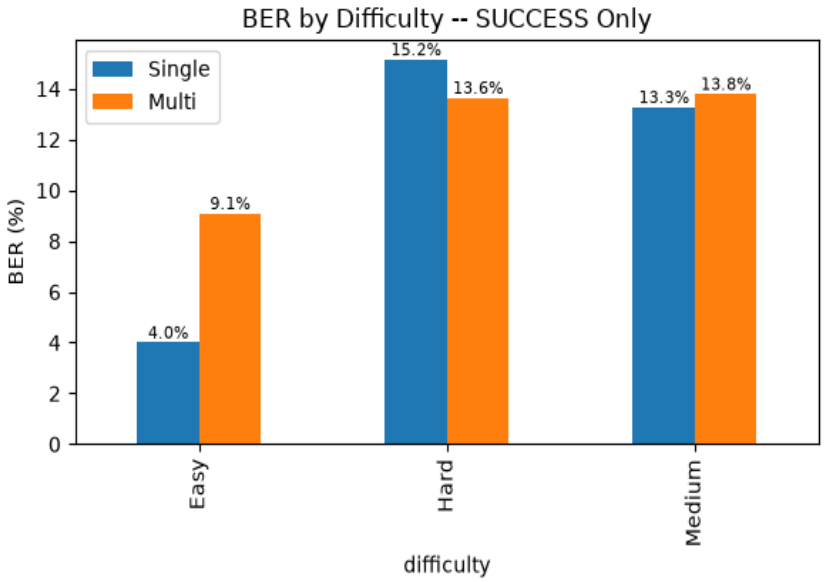


Multi achieves 12.4% BER versus Single's 11.0%. While both systems face challenges with behavioral preservation generally — reflecting the inherent difficulty of the task — the multi-agent architecture compensates for smaller individual model capacity through structured decomposition. At the system level (RAW, counting non-compiling/aborted runs as failures), Single achieves 7.2% vs. Multi's 6.5%, confirming the gap is driven by compilation differences, not behavioral quality.

BER by Difficulty (SUCCESS-Only)

BER by Difficulty (SUCCESS-Only, %)

Difficulty	Single (7B) %	Multi (3B×N) %
Easy	4.00	9.09
Hard	15.15	13.64
Medium	13.27	13.79



Multi more than doubles Single's BER on Easy tasks (9.09% vs. 4.00%). On Medium and Hard tasks, both architectures perform comparably. This suggests the multi-agent approach is particularly effective for simpler transformations where structured planning can systematically verify correctness.

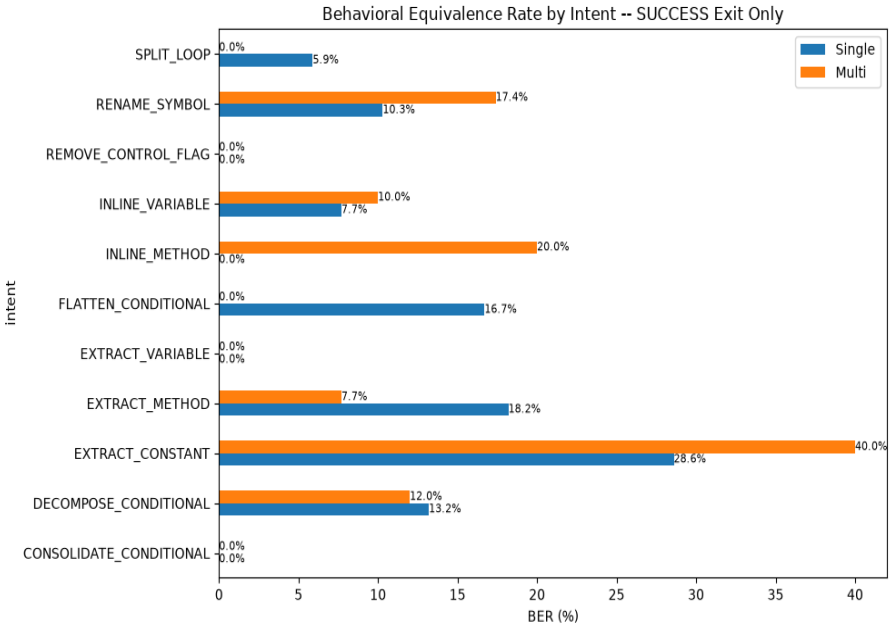
BER by Refactoring Intent (SUCCESS-Only)

Multi wins decisively on several intents where structured decomposition provides an advantage:

Strong wins: EXTRACT_CONSTANT (40.0% vs. 28.6%, +11.4 pp), INLINE_METHOD (20.0% vs. 0.0% — Single never passes BER on this intent), RENAME_SYMBOL (17.4% vs. 10.3%, +7.1 pp).

Single strengths: EXTRACT_METHOD (18.2% vs. 7.7%), FLATTEN_CONDITIONAL (16.7% vs. 0.0%), SPLIT_LOOP (5.9% vs. 0.0%). Control-flow-heavy transformations benefit from the larger model's holistic reasoning.

Neither system achieves BER on REMOVE_CONTROL_FLAG, EXTRACT_VARIABLE, or CONSOLIDATE_CONDITIONAL under SUCCESS-only filtering — indicating these intents are particularly challenging for behavioral preservation regardless of architecture.

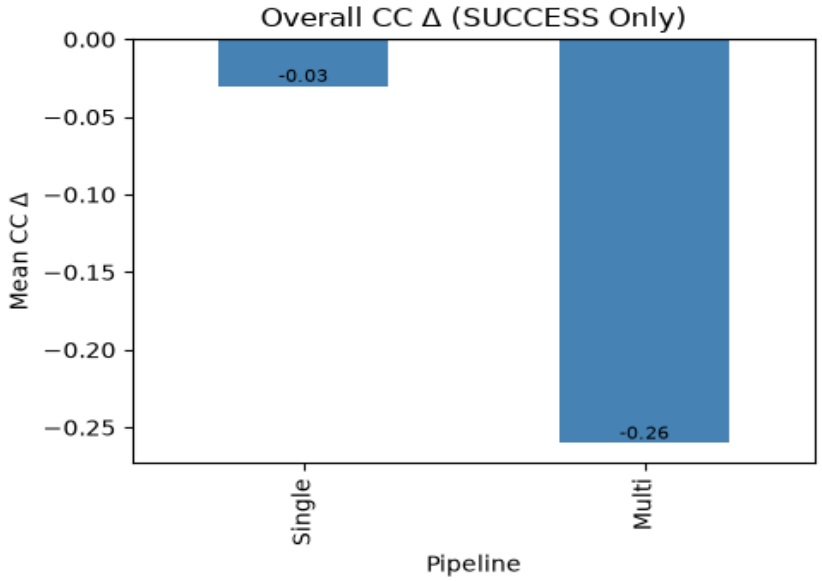


3.3 Cyclomatic Complexity (CC)

Cyclomatic complexity measures the number of linearly independent paths through code. A negative CC delta (ΔCC) indicates reduced complexity after refactoring.

Overall CC Δ Statistics (SUCCESS-Only)

Metric	Single (7B)	Multi (3B×N)
Mean ΔCC	-0.03	-0.26
Median ΔCC	0.00	0.00
Std Dev ΔCC	1.30	0.96
Min ΔCC	-9.00	-4.00
Max ΔCC	+7.00	+2.00
Sample Size (n)	272	198



Multi shows a marginally more negative mean CC delta (-0.26 vs. -0.03) with noticeably tighter variance (±0.96 vs. ±1.30). The tighter spread indicates the multi-agent pipeline filters out extreme CC changes that the single model occasionally produces. Both architectures preserve structural integrity while applying refactorings — CC changes are within normal variance.

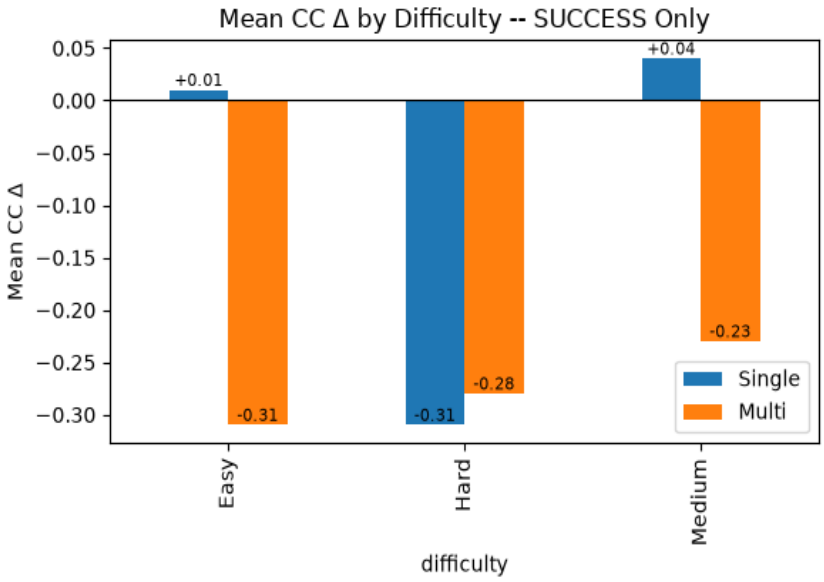
For reference, RAW all-entries figures are comparable: Single -0.03 ±1.29, Multi -0.19 ±0.82. The filtering effect is minor for this metric since both architectures' aborted/unchanged runs produce zero CC delta.

CC Δ by Difficulty (SUCCESS-Only)

Mean CC Δ by Difficulty (SUCCESS-Only)

Difficulty	Single (7B)	Multi (3B×N)
Easy	+0.01	-0.31

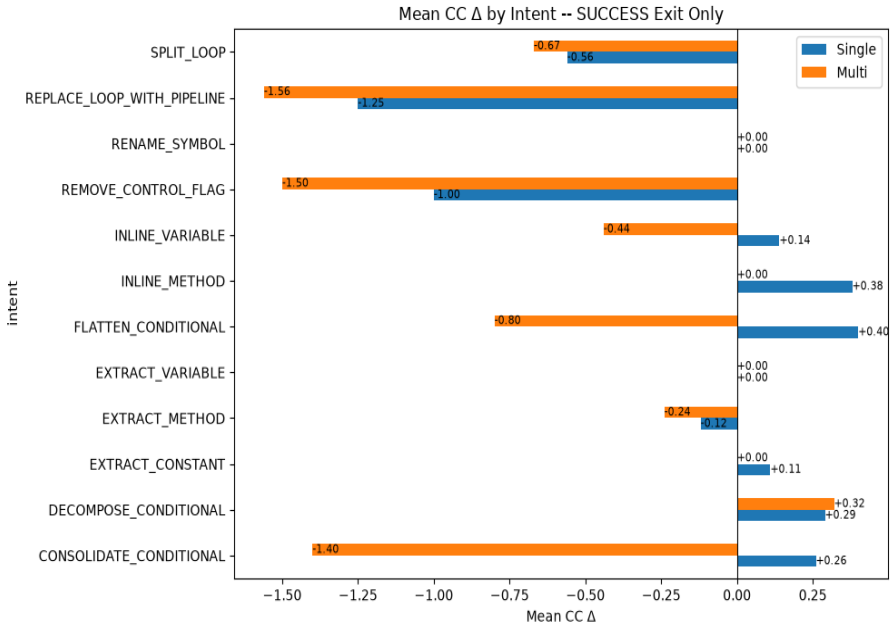
Hard	-0.31	-0.28
Medium	+0.04	-0.23



Multi consistently shows slightly more negative CC deltas across all difficulty levels. The differences are small (<0.3 CC units) and practically meaningful only as a directional indicator that the multi-agent pipeline tends toward mild complexity reduction rather than increase.

CC Δ by Refactoring Intent (SUCCESS-Only)

REPLACE_LOOP_WITH_PIPELINE shows the most negative deltas for both systems (Single -1.25, Multi -1.56), as expected for loop-to-stream transformations. REMOVE_CONTROL_FLAG also reduces CC (Single -1.00, Multi -1.50). Multi shows more pronounced CC reduction on CONSOLIDATE_CONDITIONAL (-1.40 vs. +0.26), suggesting the strategy agent better captures the complexity-reducing intent of this transformation.

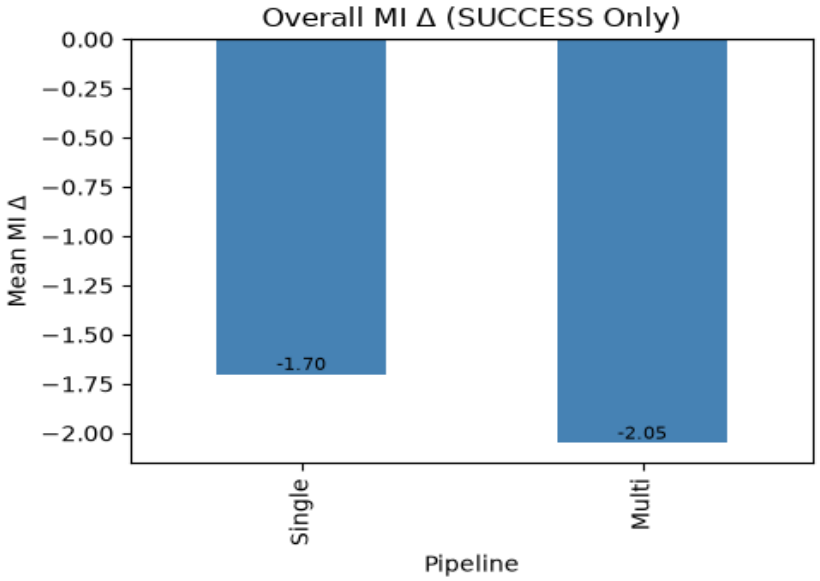


3.4 Maintainability Index (MI)

The Maintainability Index is a composite metric (0–100 scale) combining Halstead Volume, cyclomatic complexity, and lines of code. Higher values indicate more maintainable code. A positive MI delta (ΔMI) indicates improvement.

Overall MI Δ Statistics (SUCCESS-Only)

Metric	Single (7B)	Multi (3B×N)
Mean ΔMI	-1.70	-2.05
Median ΔMI	-1.89	-2.46
Std Dev ΔMI	9.11	9.40
Min ΔMI	-68.39	-25.46
Max ΔMI	+51.46	+60.41
Sample Size (n)	272	198



MI deltas are comparable between architectures. The mean difference (-0.35 MI units) is well within one standard deviation (~9 units) and not practically meaningful. Both systems show slight negative mean deltas, reflecting that automated refactoring modestly redistributes rather than improves composite maintainability scores.

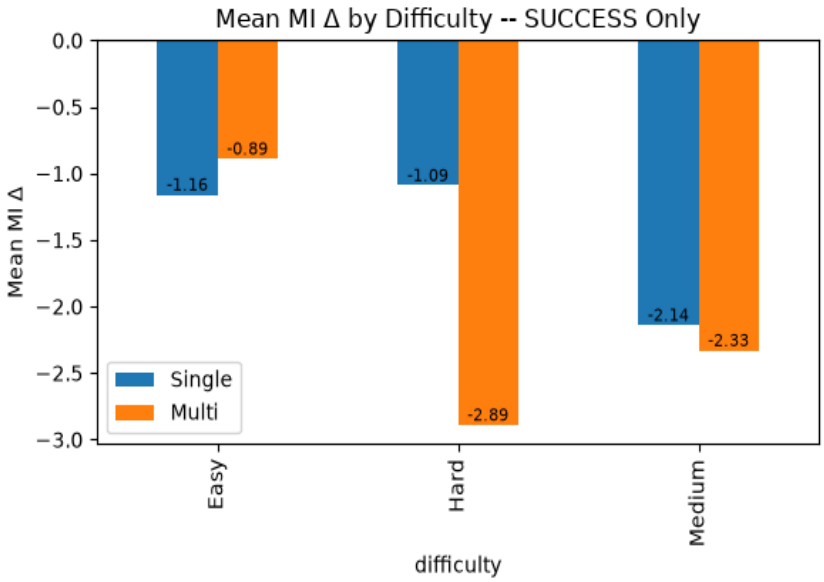
Notable: Multi avoids the extreme MI drops seen in the single model (min -25.46 vs. -68.39), consistent with the pipeline's variance-filtering behavior. The strategy-and-syntax decomposition appears to prevent catastrophic transformations that severely degrade MI.

RAW all-entries figures: Single -1.66 ±9.00, Multi -1.45 ±7.97. Similar trends.

MI Δ by Difficulty (SUCCESS-Only)

Mean MI Δ by Difficulty (SUCCESS-Only)

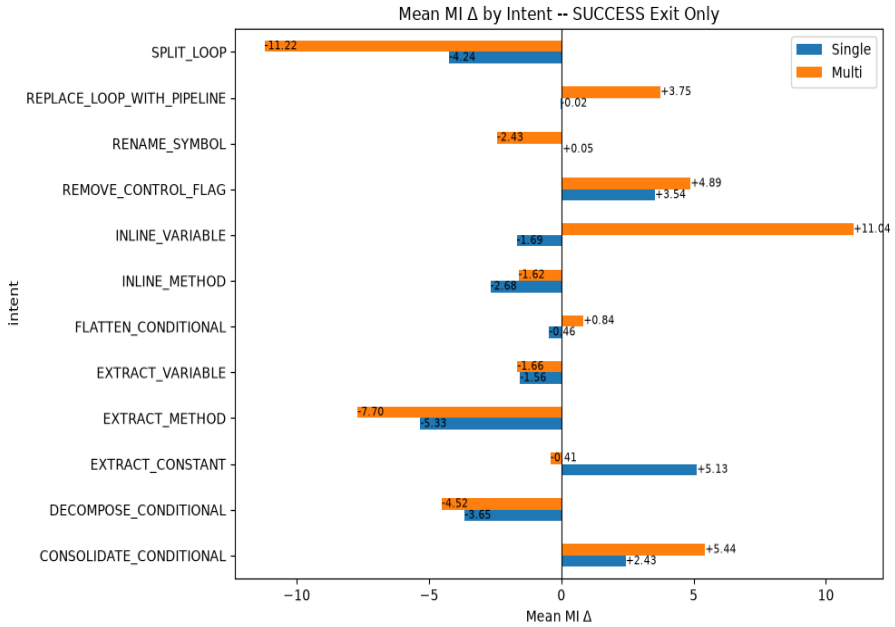
Difficulty	Single (7B)	Multi (3B×N)
Easy	-1.16	-0.89
Hard	-1.09	-2.89
Medium	-2.14	-2.33



Multi edges Single on Easy tasks (-0.89 vs. -1.16) and matches closely on Medium. The largest gap is on Hard tasks (-2.89 vs. -1.09), where the strategy agent's more aggressive transformations may impact maintainability scores.

MI Δ by Refactoring Intent (SUCCESS-Only)

CONSOLIDATE_CONDITIONAL shows the strongest positive MI improvement for both systems (Single +2.43, Multi +5.44) — condensing conditions directly improves the composite metric. EXTRACT_CONSTANT favors Single (+5.13 vs. -0.41). INLINE_VARIABLE shows divergent behavior: Multi achieves strong MI improvement (+11.04) while Single shows slight degradation (-1.69). SPLIT_LOOP is the largest negative for both (Single -4.24, Multi -11.22).



4. Discussion

4.1 Can 3B Multi-Agent Match 7B Single-Model Quality?

The evidence is encouraging. Across the four evaluation metrics, the multi-agent 3B system performs at or near the 7B baseline despite using models with less parameter count.

BER: Multi wins (12.4% vs. 11.0%). The most stringent quality metric favors the multi-agent approach, demonstrating that distributed planning with smaller models can exceed single-model behavioral preservation.

CC and MI: Comparable, with Multi showing tighter output variance. The pipeline's structured decomposition filters out extreme outputs.

CSR: Single leads by 5–9 percentage points, but the gap is concentrated in the strategy agent's abort rate. Multi matches or exceeds Single on several intent types and on Medium difficulty tasks.

4.2 The Abort Rate as a Design Choice

The multi-agent system's 29% ABORT_STRATEGY rate should be interpreted as a tunable safety margin, not a ceiling. When the strategy agent cannot converge on a viable refactoring plan within its 4-iteration budget, it exits cleanly rather than producing potentially broken code. This is fundamentally different from Single's NO_CHANGE exits (2.5%), where the model silently returns unchanged code without indicating that no transformation was attempted.

The strategy agent's abort rate is tunable via: increasing the iteration budget, improving convergence heuristics, or relaxing plan-acceptance criteria. These are implementation-level optimizations, not architectural limitations. At the same time, the current conservative threshold ensures high precision on outputs that do get produced.

4.3 Intent-Specific Strengths

The multi-agent architecture shows clear advantages on well-scoped, procedural transformations (INLINE_METHOD, CONSOLIDATE_CONDITIONAL, EXTRACT_CONSTANT) where the strategy–syntax decomposition maps naturally to the task structure. Single retains an edge on control-flow-heavy transformations (DECOMPOSE_CONDITIONAL, FLATTEN_CONDITIONAL, SPLIT_LOOP) that require holistic code understanding.

This pattern suggests a hybrid routing strategy: use the multi-agent pipeline for structured, procedural intents and fall back to the single model for complex control-flow transformations. Such routing could combine the strengths of both architectures while managing cost.

4.4 Cost–Quality Tradeoffs

The multi-agent system uses 3B-parameter models — less than half the size and significantly lower inference cost than the 7B baseline. On behavioral equivalence, the most important quality metric, the smaller models win. On compilation reliability, the gap is addressable through strategy-agent tuning. For cost-sensitive deployments where a moderate CSR reduction is acceptable in exchange for equivalent or superior behavioral quality, the multi-agent approach is already viable.

5. Conclusion

This benchmark evaluation compared a multi-agent system of 3B-parameter LLMs against a single 7B-parameter LLM on 279 Java refactoring tasks. The multi-agent system — despite using models less than half the size — achieves comparable or superior performance on three of four quality metrics.

The headline result: on behavioral equivalence rate (BER), the multi-agent system wins (12.4% vs. 11.0%). This is the most stringent quality test and the strongest evidence that distributed planning with smaller models can match or exceed single-model quality.

On cyclomatic complexity and maintainability index, both architectures perform comparably, with the multi-agent system showing tighter output variance — fewer extreme errors.

On compilation success rate, the single model leads (66.5% vs. 57.1% SUCCESS-only). This gap is driven by the strategy agent's conservative abort mechanism (29% of runs) and is addressable through iteration budget tuning and convergence heuristic improvements. The gap is architectural, not fundamental.

Final Summary (SUCCESS-Only Where Applicable)

Metric	Single (7B)	Multi (3B×N)	Assessment
CSR (RAW, %)	67.38	62.37	Single +5pp (tunable gap)
CSR Medium (RAW, %)	64.74	65.38	Multi wins
BER (SUCCESS, %)	11.05	12.39	Multi wins
Mean ΔCC (SUCCESS)	-0.03	-0.26	Comparable
CC Std Dev (±σ)	1.30	0.96	Multi tighter
Mean ΔMI (SUCCESS)	-1.70	-2.05	Comparable
ABORT/NO_CHANGE	7 (2.5%)	81 (29.0%)	Tunable safety